

SPM: Workload Managers (SLURM)

Academic year 2025-2026
Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Resource Management Systems in compute clusters
- The SLURM resource manager: overview and simple commands

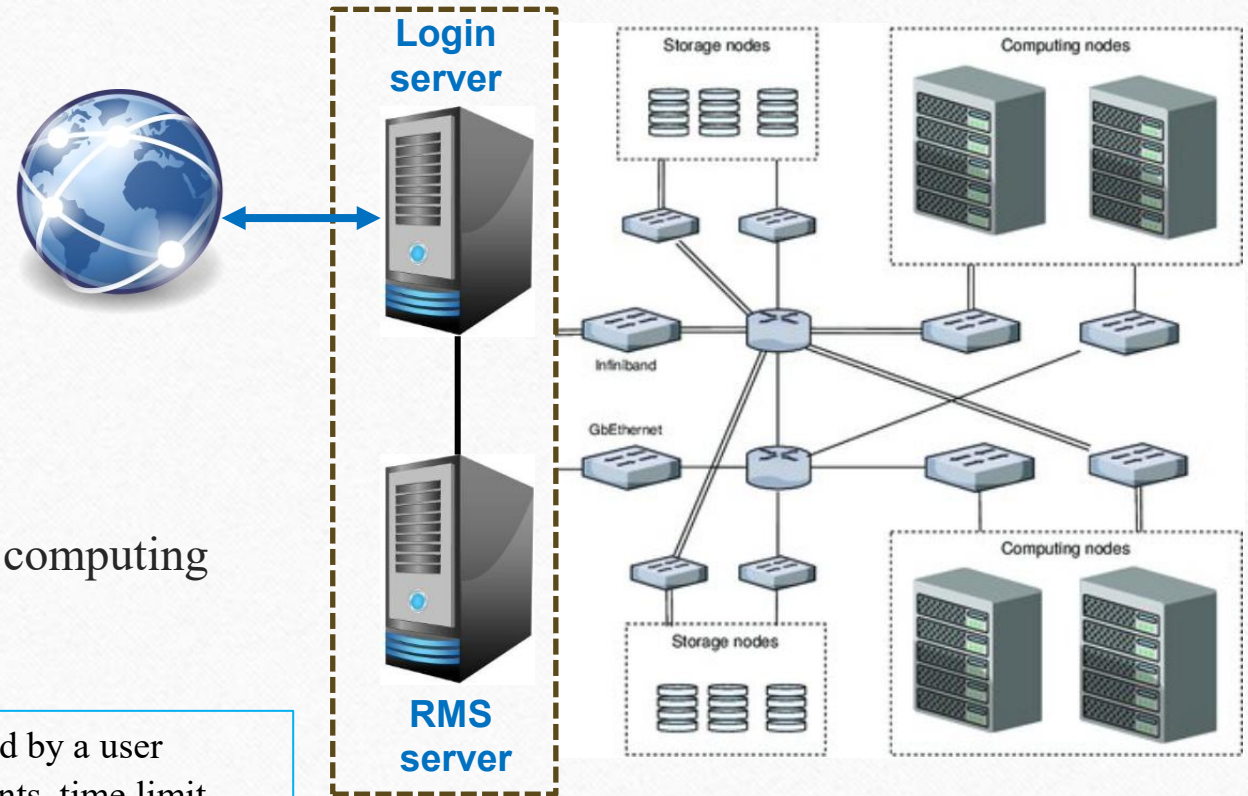


Resource Management Systems

- Resource Management Systems (**RMSs**) are software systems that:
 - Allocate and schedule computing resources (CPUs, memory, GPUs) in a multi-user, multi-node environment
 - Manage users' **jobs**, i.e., their submission for execution (accept, queue, start, and monitor jobs)
 - Optimize resource utilization and fair allocation of such resources among all users
 - Users do not run heavy jobs on login servers
- They are extensively used in HPC clusters, cloud computing environment and data centers
- Examples: **SLURM**, **PBS**, **LSF**, **SGE**

What is a job? A **job** is a unit of work submitted by a user

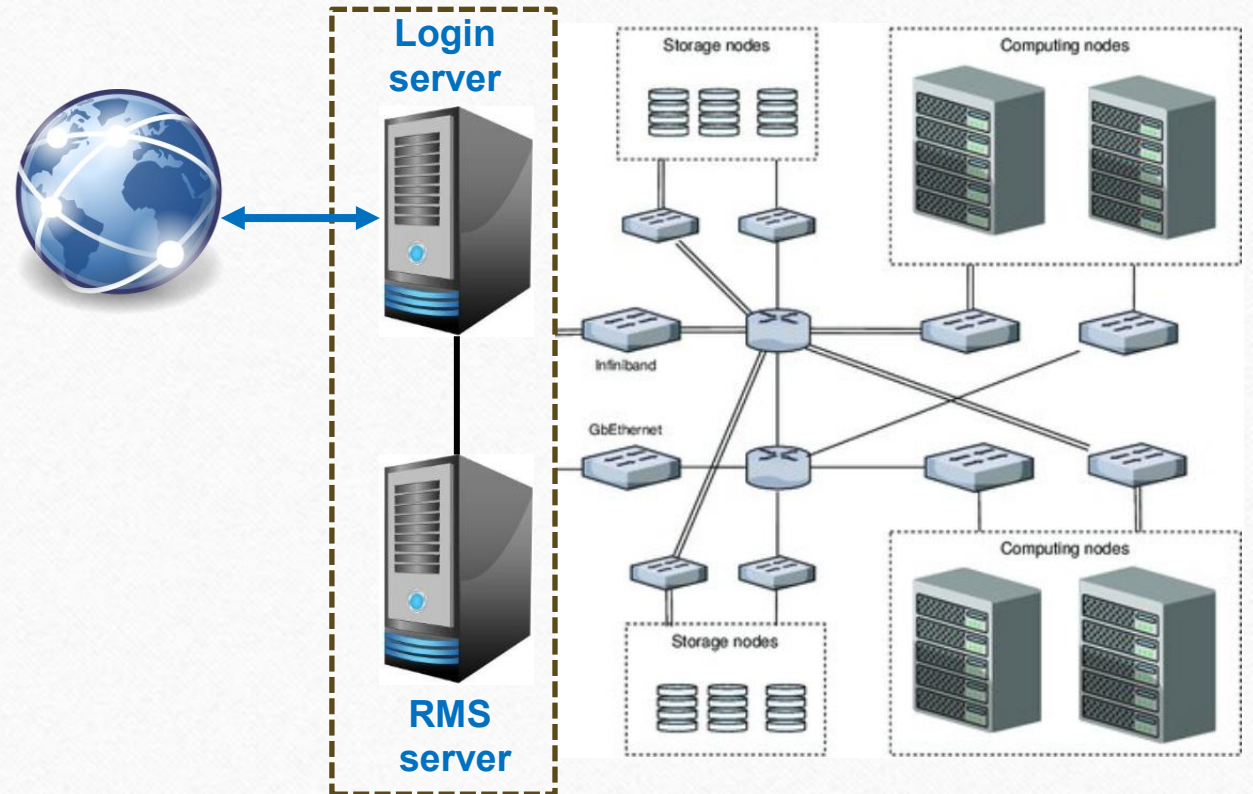
- Executable, input data, resource requirements, time limit
- It can be batch or interactive



HPC infrastructure

Resource Management Systems (cont.)

- Managing the resources of a supercomputer with thousands of nodes is NOT trivial
- The nodes of a cluster are divided into **login** (entry) nodes, **RMS** nodes, and **computational** nodes.
 - Usually, there is more than one login node
- To access the compute nodes, you should know which Resource Manager/Job Scheduler is used
- In our small course cluster, we use **SLURM**



HPC infrastructure

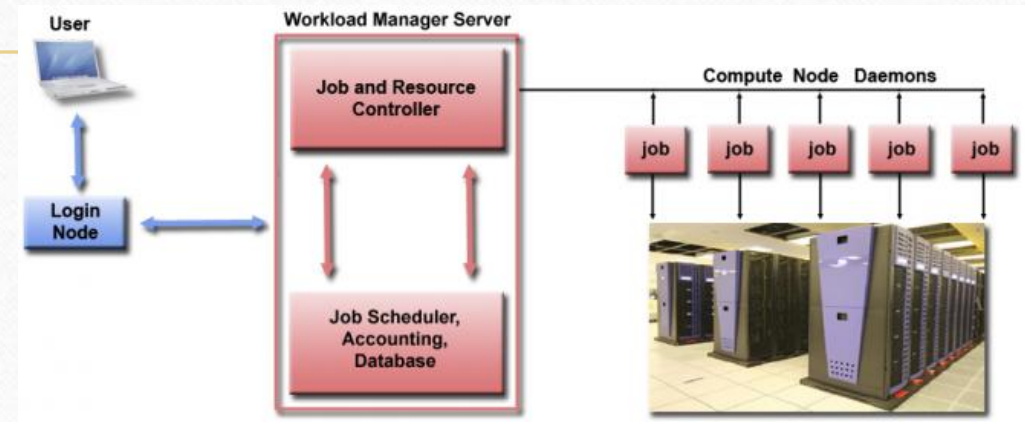
Resource Management in HPC clusters

- **Resource Allocation**

- Assign nodes to users
- Static vs dynamic allocation
- Granularity: a set of whole nodes, a set of cores of a single node

- **Manage the schedule**

- Decide when a given job starts on the assigned nodes and relinquish the resources at the end of program execution
- FCFS, Backfilling, Priority-Based, Fair-Share



- **Job placement and launch**

- Dispatch jobs to allocated nodes
- Launch parallel jobs across allocated nodes, e.g., MPI processes

- **Monitoring and Accounting**

- Display running and waiting jobs
- Account for resource utilization

SLURM

- **SLURM** is an open-source job scheduler and resource manager for HPC clusters
 - Simple Linux Utility for Resource Management ([SLURM](#))
 - developed starting from 2001 at the Lawrence Livermore National Laboratory
- Manages the allocation of computing resources for parallel and batch jobs
- **Computational nodes are grouped into partitions** (logical sets of resources)
- Partitions can be configured in different ways:
 - limiting the number of hours and nodes per job
- Resources can be allocated on demand with the **salloc/srun** commands
- It supports job dependencies
- **sinfo** to know which partitions are available on the cluster.

```
[torquati@spmln ~]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
normal*    up       30:00      2  alloc node[01-02]
normal*    up       30:00      6  idle  node[03-08]
gpu-shared up       10:00      1  idle  node09
gpu-excl   up       30:00      1  idle  node09
[torquati@spmln ~]$ squeue
              JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
              538596   normal    sleep  torquati  R       0:15      2 node[01-02]
[torquati@spmln ~]$
```


SLURM policies

- SLURM allocates requested resources within the selected partition, if available
- Jobs are scheduled by priority (highest priority first)
- Priority can depend on multiple factors:
 - Job size and resources requested
 - Job age (i.e., waiting time in the queue)
 - User fair-share usage (to prevent a single user or group from monopolizing the partition/system)
 - Partition priority (some partitions may have higher priority)
 - Requested time limit (walltime)
 - QoS/reservation (if enabled)
 - Account/project limit (site policy)
 - Dependencies (a job becomes eligible to start only after its dependencies)
- You can inspect job priority and the reason for waiting (pending reason)
 - `sprio -j <jobid>`
 - `squeue -j <jobid> -o %R`

SLURM policies (cont.)

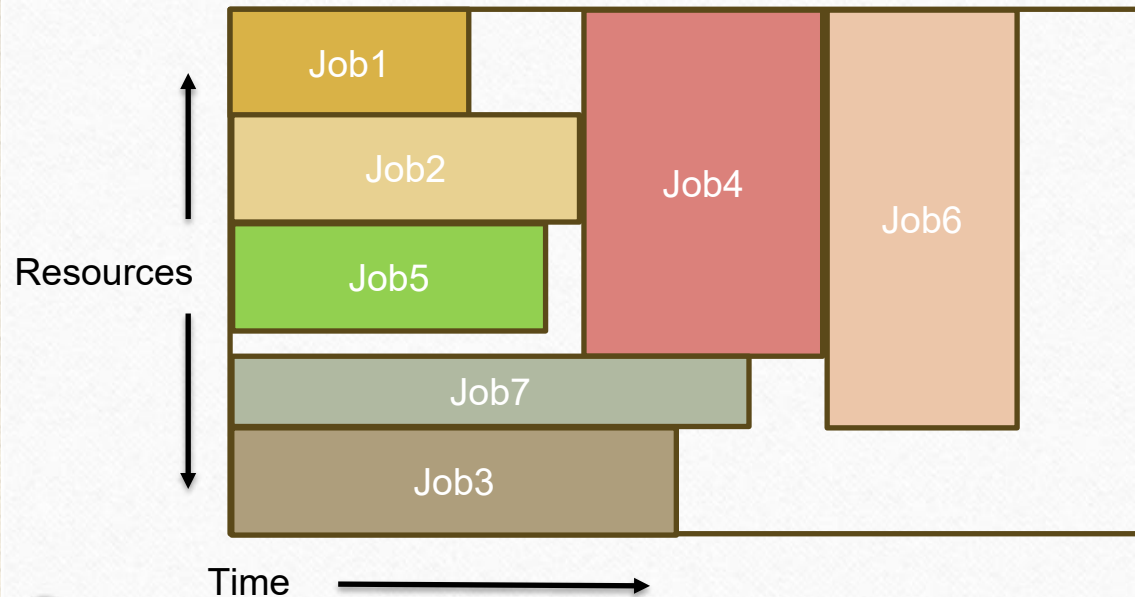
- Scheduling behavior:
 - **priority ordering**: SLURM considers jobs in priority order, subject to partition constraints
 - Priority is used for time-sensitive or reserved workloads
 - **backfill algorithm**: opportunistic scheduling to fill gaps
 - It runs lower-priority jobs if they don't delay higher-priority ones
 - It helps to increase cluster utilization by filling in gaps
- Typically, SLURM ranks jobs by priority, then uses backfilling to start additional jobs that fit without delaying higher-priority jobs
- **Job's requested time limit is very important and affects scheduling decisions, especially backfilling**
 - Overestimating reduces opportunities to start earlier
 - Underestimating risks job termination at the time limit

Priority queue algorithm

- Jobs are evaluated globally, subject to partition constraints and priorities
 - the highest-priority job gets executed first
- Each job is assigned a priority value based on factors such as:
 - fairshare (users with lower recent usage get higher priority)
 - age (older jobs gain priority over time)
 - size can be a scheduling factor depending on site policy (larger jobs may be prioritized)
 - QoS and partition configuration
- Queue Management: Jobs are placed in a queue and sorted by priority
 - Preemption: if enabled, lower-priority jobs may be preempted to make room for higher-priority jobs
- Potential issue:
 - Blocking: if resources are insufficient, the highest-priority job may delay other jobs

Backfill algorithm, example

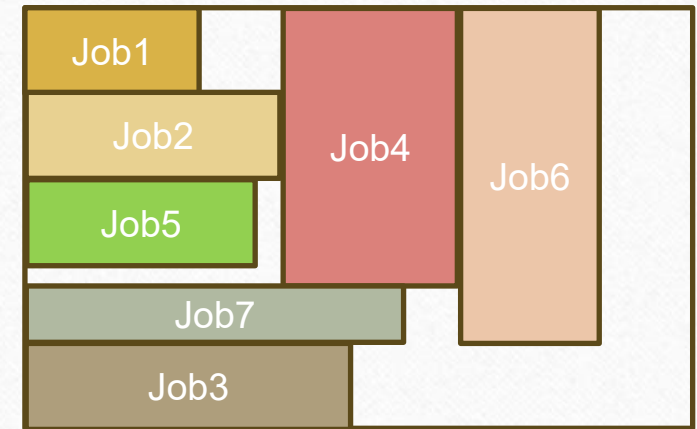
- The default algorithm attempts to schedule low-priority jobs if they do not prevent higher-priority jobs from starting at the scheduled time
 - **Goal:** fill in gaps in the schedule without delaying reserved resources for higher-priority jobs.



- All jobs arrive at the same time (t_0)
- Jobs 1,2 and 3 start immediately
- Job 4 cannot start. It waits for Job 2 termination because needs the same resources.
- Job 5 can start immediately if it finishes no later than Job 2 (ensuring Job 4 is not delayed)
- Job 6 must wait until Job 4 completes
- Job 7 can start immediately using the available resources, provided it completes before Job 6 starts

Backfill algorithm

- Advantages of backfilling
 - Maximize cluster utilization preventing some resources from being idle
 - Improve system throughput by executing «small» jobs earlier
 - Does not affect high-priority jobs, thus ensuring fairness
- Disadvantages
 - **Relies on accurate job time estimates**
 - Often users overestimate job duration, leading to inefficiencies
 - If one requests 24h for a 10 min run, backfill opportunities shrink a lot
 - May starve some jobs
 - Low-priority jobs may keep getting backfilled and delayed indefinitely



spmcluster SLURM configuration

- One single frontend/login node (spmcluster.unipi.it)
 - SLURM runs on the login node
- 9 compute nodes connected through **10 Gigabit Ethernet**
 - The login node is also connected to the 10 Gigabit switch
- **Three SLURM queues** (three partitions -- sinfo)
- Scheduler: backfill
 - `scontrol show config | grep -i schedulertype`
- **Only one job per node** (Oversubscribe=Exclusive)
- Users' homes exported on all compute nodes via NFS (Network File System)
- Time limit 30 minutes per job
 - `squeue -h -j $SLURM_JOBID -o %L // time left for my job with SLURM_JOBID id`

To gain access to the spmcluster resources, please read the document [spmcluster-access.pdf](#)

Main SLURM commands

- **srun**: executes a parallel job:
 - Usage: **srun** [options] [executable] [argument]
- some common options:
 - **--nodes** / **-N**: the number of computational nodes to use
 - **--ntasks** / **-n**: the total number of tasks spawned from the job
 - **--cpus-per-task**: the number of CPUs per task on the same node to use
 - **--ntasks-per-core**: the number of tasks per core to use
 - **--ntasks-per-socket**: the number of tasks per socket to use
 - **--ntasks-per-node**: the number of tasks spawned per node
 - **--exclusive**: ensures allocated nodes are not shared with other jobs
 - **--time**: specifies the time requested (default: 00:30:00 time)
 - **--partition**: specifies the partition to use (default: normal)

Understanding SLURM terminology:

- **Node**: physical compute server
- **Task**: instance of a running program
- **CPU**: hardware thread or core

Simple *srun* examples

- `srun hostname`
- `srun -N 3 hostname`
- `srun -N 2 -n 4 --cpus-per-task=2 python3 test.py`
- `srun make`
- `srun --nodelist=node03 --ntasks=1 --cpus-per-task=4 ./test`
- The internal nodes have 2 SMP CPUs, each with 8 cores 2-way hyperthreading

NUMA:

```
NUMA node(s):          2
NUMA node0 CPU(s):     0-7,16-23
NUMA node1 CPU(s):     8-15,24-31
```

```
[torquati@spmln SLURM]$ hostname
spmln
[torquati@spmln SLURM]$ srun hostname
node01
[torquati@spmln SLURM]$ srun -N 3 hostname
node03
node02
node01
[torquati@spmln SLURM]$ srun -N 2 -n 4 --cpus-per-task=2 python3 test.py
Hostname: node02
Cores where I can run: 0, 16
Hostname: node02
Cores where I can run: 8, 24
Hostname: node01
Cores where I can run: 8, 24
Hostname: node01
Cores where I can run: 0, 16
[torquati@spmln SLURM]$ srun make
g++ -O3 test.cpp -o test
[torquati@spmln SLURM]$ srun --nodelist=node03 --ntasks=1 --cpus-per-task=4 ./test
Hostname: node03
Cores where I can run: 0 1 16 17
[torquati@spmln SLURM]$
```


Other useful commands

- **sbatch**: syntax `sbatch [options] [script]`
 - submits a batch script to SLURM for non-interactive executions
 - `sbatch my_job_script.sh`
- **salloc**: syntax `salloc [options] [command] [arguments]`
 - allocates compute nodes for an interactive session
 - `salloc -N 1 --time=00:15:00`
- **squeue**:
 - displays the status of jobs currently in the SLURM queue
 - `squeue -u $USER` (`squeue --me`) displays only my jobs
 - `squeue -j <jobid>` displays a specific jobid
- **scancel**:
 - forces the termination of a job
 - `scancel -u $USER -state=PENDING` cancels only my pending jobs
- **scontrol**: provides information on the system
 - `scontrol show nodes`: provides information on available nodes
 - `scontrol show jobs`: provides information on running jobs

SLURM interactive shell

- Users cannot ssh to internal nodes (possible only for root)
- An interactive shell on a node can be started with the following command:

```
srun -n 1 --time=00:20:00 --pty bash -i
```

- Alternatively:

```
salloc -w node07 --time=00:20:00  
ssh node07  
scancel $SLURM_JOBID
```

```
[torquati@spmln ~]$ ssh node07  
Access denied: user torquati (uid=1001) has no active jobs on this node.  
Connection closed by 172.16.40.7 port 22  
[torquati@spmln ~]$ salloc -w node07 --time=01:00:00  
salloc: Granted job allocation 50561  
[torquati@spmln ~]$ ssh node07  
Last login: Sat Feb  1 18:18:38 2025 from 172.16.40.254  
[torquati@node07 ~]$ exit  
logout  
Connection to node07 closed.  
[torquati@spmln ~]$ squeue --me  
          JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)  
          50561    normal interact torquati  R        0:11      1 node07  
[torquati@spmln ~]$ scancel --me  
salloc: Job allocation 50561 has been revoked.  
Hangup  
[torquati@spmln ~]$ squeue  
          JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)  
[torquati@spmln ~]$
```


Example SLURM job file

```
#!/bin/bash
```

```
#SBATCH -p normal
```

```
#SBATCH -N 4
```

```
#SBATCH -o %j.log
```

```
#SBATCH -e %j.err
```

```
echo $SLURM_JOB_NODELIST
```

```
srunch /bin/hostname
```

```
[torquati@spmln spmcode9]$ sbatch basic_sbatch.sh
Submitted batch job 244
[torquati@spmln spmcode9]$ cat 244.log
node[01-04]
node01
node02
node03
node04
```

```
#!/bin/bash
```

```
#SBATCH --job-name=my_job # Job name
```

```
#SBATCH --output=output.log # Standard output log file
```

```
#SBATCH --error=error.log # Standard error log file
```

```
#SBATCH --time=01:00:00 # Time limit (hh:mm:ss)
```

```
#SBATCH --nodes=2 # Number of nodes
```

```
#SBATCH --ntasks=4 # Number of total tasks (MPI processes)
```

```
#SBATCH --cpus-per-task=2 # Number of CPU cores per task
```

```
#SBATCH --partition=normal # Partition to submit to
```

```
#SBATCH --mem=4G # Memory per node (4GB)
```

```
srunch make myprog
```

```
srunch ./myprog param1 param2 ...
```


Working with MPI in a SLURM cluster

- Use the version configured and installed in your system.
- In the `spmcluster` the MPI implementation installed is `OpenMPI v5.0.3`
 - `ompi_info` to see configuration information
 - OpenMPI v5 documentation: <https://docs.open-mpi.org/en/v5.0.x/>
- MPI directory: `/opt/ohpc`
 - `/opt/ohpc/pub` directory
- In general, on different systems, MPI can be configured in different ways (e.g., with CUDA, etc.)
- SLURM and MPI:
 - `srun --mpi=pmix --nodes N myprog` // using PMIx plugin – Process Management Interface for Exascale
 - `salloc --nodes N --time 00:01:00` // allocating resources and then using `mpirun`
`mpirun -n N myprog`
- MPI application can be compiled with: `mpicc/mpicxx`

We will discuss how to run MPI programs with SLURM later, after introducing MPI

figured in different

Suggested Readings

- SLURM documentation:
 - <https://slurm.schedmd.com/documentation.html>
- SLURM Tutorial:
 - <https://hpc.llnl.gov/banks-jobs/running-jobs/slurm>